

■■ NaijaHelp — Full-Stack Study Guide

Goal: Build a Nigerian local-services and emergency finder using HTML, CSS and JavaScript first, then connect it to a real backend with Node.js, Express and Supabase.

Recommended learning order: HTML → CSS → JavaScript → DOM → Fetch/API → Node.js → Express → Supabase/PostgreSQL → Authentication → Deployment.

Important: The commands below are for study and building. Do not paste everything at once. Build and test each stage before moving to the next.

1. Final project vision

NaijaHelp lets users find useful services such as hospitals, pharmacies, police stations, fire stations, mechanics, electricians, plumbers and other local providers. Users can search, filter, view details, contact a provider, and eventually use accounts and an admin dashboard.

Core pages: Home, Services, Service Details, Login/Register, Add Service, User Dashboard, Admin Dashboard, About/Contact.

2. Project folder structure

```
naijahelp/  
  ■■■ frontend/  
    ■ ■■■ index.html  
    ■ ■■■ services.html  
    ■ ■■■ login.html  
    ■ ■■■ register.html  
    ■ ■■■ dashboard.html  
    ■ ■■■ css/  
      ■ ■■■ style.css  
    ■ ■■■ js/  
      ■ ■■■ app.js  
      ■ ■■■ services.js  
      ■ ■■■ auth.js  
    ■ ■■■ assets/  
  ■■■ backend/  
    ■■■ src/  
      ■ ■■■ server.js  
      ■ ■■■ routes/  
      ■ ■■■ controllers/  
      ■ ■■■ middleware/  
      ■ ■■■ config/  
    ■■■ .env  
    ■■■ package.json  
    ■■■ .gitignore
```

3. HTML commands/concepts to study

HTML is the structure of your website. Learn these elements before adding JavaScript.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>NaijaHelp</title>  
  <link rel="stylesheet" href="css/style.css">  
</head>  
<body>  
  <header>  
    <h1>NaijaHelp</h1>  
    <nav>  
      <a href="index.html">Home</a>  
      <a href="services.html">Services</a>
```

```

    </nav>
</header>

<main>
  <section>
    <h2>Find Help Near You</h2>
    <input id="searchInput" type="search" placeholder="Search services...">
    <button id="searchBtn">Search</button>
  </section>

  <section id="serviceList"></section>
</main>

<script src="js/app.js"></script>
</body>
</html>

```

Study: headings, paragraphs, links, images, forms, inputs, buttons, sections, divs, semantic tags, IDs and classes.

4. CSS commands/concepts to study

```

/* Select an element */
body {
  margin: 0;
  font-family: Arial, sans-serif;
}

/* Class */
.service-card {
  padding: 20px;
  border-radius: 12px;
}

/* ID */
#searchInput {
  width: 100%;
}

/* Flexbox */
.service-list {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

/* Grid */
.service-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

/* Responsive design */
@media (max-width: 768px) {
  .service-grid {
    grid-template-columns: 1fr;
  }
}

```

Study: selectors, box model, margin, padding, border, width/height, colors, fonts, display, Flexbox, Grid, position, hover effects and media queries.

5. JavaScript fundamentals

```

let serviceName = "Jalingo Specialist Hospital";
const serviceType = "Hospital";
let serviceCount = 10;

console.log(serviceName);
console.log(serviceType);
console.log(serviceCount);

```

```
function greetUser(name) {
  return "Welcome " + name;
}

console.log(greetUser("Ephraim"));

if (serviceCount > 0) {
  console.log("Services available");
} else {
  console.log("No services found");
}
```

Study: let, const, strings, numbers, booleans, arrays, objects, functions, if/else, loops, template literals, arrow functions, map, filter, find and JSON.

6. Create your first service data

```
const services = [
  {
    id: 1,
    name: "Jalingo Specialist Hospital",
    category: "Hospital",
    location: "Jalingo",
    phone: "08000000000"
  },
  {
    id: 2,
    name: "ABC Auto Repairs",
    category: "Mechanic",
    location: "Jalingo",
    phone: "08000000001"
  }
];

console.log(services);
```

7. Display data dynamically

```
const serviceList = document.getElementById("serviceList");

function displayServices(data) {
  serviceList.innerHTML = "";

  data.forEach(service => {
    serviceList.innerHTML += `
    <article class="service-card">
      <h3>${service.name}</h3>
      <p>${service.category}</p>
      <p>${service.location}</p>
      <a href="tel:${service.phone}">Call Now</a>
    </article>
    `;
  });
}

displayServices(services);
```

8. Search and filtering

```
const searchInput = document.getElementById("searchInput");

searchInput.addEventListener("input", () => {
  const keyword = searchInput.value.toLowerCase();

  const results = services.filter(service =>
    service.name.toLowerCase().includes(keyword) ||
    service.category.toLowerCase().includes(keyword) ||
    service.location.toLowerCase().includes(keyword)
  );

  displayServices(results);
});
```

```
});
```

9. localStorage — temporary browser storage

```
localStorage.setItem("username", "Ephraim");

const username = localStorage.getItem("username");
console.log(username);

localStorage.removeItem("username");
```

Use localStorage while learning frontend state. Once the project has a real backend, important user data should be stored securely on the server/database instead.

10. Start the backend

Install Node.js first. Then open a terminal inside the backend folder.

```
mkdir backend
cd backend
npm init -y

npm install express cors dotenv
npm install @supabase/supabase-js
npm install bcryptjs jsonwebtoken
npm install helmet express-rate-limit
npm install --save-dev nodemon
```

11. Backend package scripts

```
"scripts": {
  "start": "node src/server.js",
  "dev": "nodemon src/server.js"
}

npm run dev
```

12. Basic Express server

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const app = express();

app.use(cors());
app.use(express.json());

app.get("/api/health", (req, res) => {
  res.json({ status: "ok" });
});

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Test in your browser: <http://localhost:3000/api/health>

13. Environment variables

```
# backend/.env

PORT=3000
SUPABASE_URL=your_supabase_project_url
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
JWT_SECRET=make_a_long_random_secret
```

Security: Never upload .env to GitHub. Add it to .gitignore. Never put a Supabase service-role key in frontend JavaScript.

14. .gitignore

```
node_modules/  
.env  
.env.local  
.env.production
```

15. Supabase database design

Create a Supabase project and a services table. A useful first schema is:

```
services  
-----  
id  
name  
category  
description  
phone  
email  
address  
city  
state  
latitude  
longitude  
is_verified  
created_at  
user_id
```

Later add users/profiles, reports, reviews, favorites and emergency contacts. Use Row Level Security (RLS) and policies rather than trusting the frontend.

16. Connect Express to Supabase

```
const { createClient } = require("@supabase/supabase-js");  
  
const supabase = createClient(  
  process.env.SUPABASE_URL,  
  process.env.SUPABASE_SERVICE_ROLE_KEY  
);  
  
module.exports = supabase;
```

17. GET services API

```
app.get("/api/services", async (req, res) => {  
  try {  
    const { data, error } = await supabase  
      .from("services")  
      .select("*")  
      .order("created_at", { ascending: false });  
  
    if (error) throw error;  
  
    res.json(data);  
  } catch (error) {  
    res.status(500).json({ error: "Failed to load services" });  
  }  
});
```

18. Frontend calls the backend

```
async function loadServices() {  
  const response = await fetch("http://localhost:3000/api/services");  
  
  if (!response.ok) {
```

```

    throw new Error("Failed to load services");
  }

  const data = await response.json();
  displayServices(data);
}

loadServices();

```

When you deploy the backend, replace the localhost URL with your deployed API URL. Better still, store the API URL in the frontend environment configuration.

19. POST — add a service

```

app.post("/api/services", async (req, res) => {
  try {
    const {
      name, category, description,
      phone, address, city, state
    } = req.body;

    if (!name || !category || !city) {
      return res.status(400).json({
        error: "name, category and city are required"
      });
    }

    const { data, error } = await supabase
      .from("services")
      .insert([
        {
          name,
          category,
          description,
          phone,
          address,
          city,
          state
        }
      ])
      .select()
      .single();

    if (error) throw error;

    res.status(201).json(data);
  } catch (error) {
    res.status(500).json({ error: "Failed to create service" });
  }
});

```

20. Authentication — learning path

For a full application, users should register/login and receive authenticated access. Study these concepts:

```

Register
↓
Validate input
↓
Hash password
↓
Save user
↓
Login
↓
Verify password
↓
Create token/session
↓
Authenticated requests

```

If using Supabase Auth, prefer its authentication system rather than inventing your own password-storage system. Your backend can verify authenticated users and enforce authorization.

21. Useful API routes for NaijaHelp

```
GET    /api/services
GET    /api/services/:id
GET    /api/services?category=Hospital
GET    /api/services?city=Jalingo

POST   /api/services
PATCH /api/services/:id
DELETE /api/services/:id

POST   /api/auth/register
POST   /api/auth/login
GET    /api/auth/me

POST   /api/reports
GET    /api/reports

POST   /api/reviews
GET    /api/services/:id/reviews
```

22. Admin dashboard

An administrator should be able to:

- View all submitted services
- Approve/reject service listings
- Mark services as verified
- Edit incorrect information
- Remove fake/duplicate listings
- Review user reports
- View basic statistics

23. Security checklist

1. Never expose service-role secrets in frontend code.
2. Validate all request data on the backend.
3. Use authentication for protected routes.
4. Use authorization/roles for admin actions.
5. Enable database Row Level Security.
6. Rate-limit sensitive endpoints.
7. Use HTTPS in production.
8. Do not trust IDs or roles sent by the browser.
9. Keep secrets in environment variables.
10. Do not commit .env to Git.

24. Location feature

For a later version, add browser geolocation so users can find nearby services. The browser asks the user for permission.

```
navigator.geolocation.getCurrentPosition(
  position => {
    const latitude = position.coords.latitude;
    const longitude = position.coords.longitude;

    console.log(latitude, longitude);
  },
  error => {
    console.log("Location permission/error:", error.message);
  }
);
```

For real distance calculations, store latitude/longitude for services and calculate distance on the backend or with a suitable geospatial/database approach.

25. Git and GitHub

```

git init
git add .
git commit -m "Initial NaijaHelp project"

git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main

```

Do not commit secrets. Check your files and .gitignore before pushing.

26. Deployment roadmap

Frontend: deploy the static frontend using a suitable static hosting service. **Backend:** deploy the Node/Express API using a Node-compatible hosting platform. **Database:** keep PostgreSQL/Supabase online. Then change the frontend API URL from localhost to your deployed backend URL.

After deployment, test health, registration/login, service listing, adding services, editing/deleting authorized records, and database policies.

27. Suggested 6-stage study plan

Stage	Learn	Build
1	HTML + CSS	Responsive NaijaHelp homepage
2	JavaScript	Search, filters, service cards
3	DOM + Fetch	Load data from an API
4	Node + Express	REST API
5	Supabase	Real database + authentication
6	Security + Deploy	Production-ready application

28. Your first assignment

Before building the backend, complete this frontend version:

```

[ ] Create index.html
[ ] Create style.css
[ ] Create app.js
[ ] Design the navigation bar
[ ] Design the search box
[ ] Create category buttons
[ ] Create service cards
[ ] Add at least 10 sample services
[ ] Make search work with JavaScript
[ ] Add category filtering
[ ] Make the page responsive
[ ] Add a service-details page
[ ] Test on your phone

```

29. Then upgrade it to the real application

```

Frontend
  ↓ fetch()
Express REST API
  ↓
Authentication / Authorization
  ↓
Supabase PostgreSQL
  ↓
Admin dashboard + user dashboard
  ↓
Deployment

```

Recommended approach: Don't start with the backend. Finish the frontend using fake JavaScript data first. Once you can search, filter and display services confidently, replace the fake array with API data from Express/Supabase. This makes it much easier to understand what each part of the full-stack application is doing.

Study this guide in order and build each section yourself. The goal is not just to copy the commands, but to understand what every command and piece of code does.